AMAZON SDE ONSITE

5-Hour Flight
Last-Minute Master Prep

Likely AWS S3 Fleet Management / Fleet Automation focus

DSA + Low-Level Design + Fleet/Distributed-Systems Revision

Important:

No preparation packet can guarantee that an interviewer will ask only these questions or guarantee a 100% result. This packet is intentionally narrow: it concentrates on the highest-value patterns for your likely team and the Amazon SDE interview format so that your limited flight time is spent on reusable ideas rather than random questions.

How to use this PDF on the flight

Flight time	What to study	Goal
0:00-0:20	Interview execution template + pattern map	Know what to say before you code.
0:20-1:45	DSA 1-4	Graphs, scheduler, Top-K, rolling window.
1:45-2:55	DSA 5-7	LRU, intervals, connected components.
2:55-3:40	DSA 8-10	TimeMap, multi-source BFS, meeting rooms.
3:40-4:40	LLD 1-3	Scheduler, deployment, reservation.
4:40-4:55	LLD 4-5	Monitoring, cache.
4:55-5:00	Final cheat sheet	Complexities, failure concepts, confidence reset.

Master Priority List

If you cannot finish everything, follow this order exactly. The first five DSA and first three LLD topics have the highest payoff for the likely Fleet Automation context.

Priority	DSA	Core pattern
P0	Course Schedule II (LC 210; LC 207 variation)	Graph + indegree + topological sort
P0	Task Scheduler (LC 621)	Max heap + cooldown queue
P0	Top K Frequent Elements (LC 347)	HashMap + min heap
P0	Rolling-window Top K (custom)	Deque + HashMap + heap selection
P0	LRU Cache (LC 146)	HashMap + doubly linked list
P1	Merge Intervals (LC 56)	Sort + scan
P1	Number of Islands (LC 200)	BFS/DFS connected components
P1	Time Based Key-Value Store (LC 981)	HashMap + binary search
P1	Rotting Oranges (LC 994)	Multi-source BFS
P1	Meeting Rooms II (LC 253)	Sorted starts/ends + two pointers

Priority	LLD	Primary pattern / idea
P0	Task / Job Scheduler	Strategy + State; priority queue
P0	Fleet Deployment Manager	Strategy + State; canary/rolling batches
P0	Host Reservation System	Strategy + atomic reservation/lease
P1	Fleet Monitoring & Alerting	Observer + Strategy
P1	Distributed Cache / LRU + TTL	Cache-aside + eviction policy

The Interview Execution Template

For every DSA problem

1. Restate the problem in your own words and confirm the input/output.
2. Ask 2-4 useful clarifying questions: constraints, duplicates, ordering, invalid input, and whether any valid answer is acceptable.
3. Walk through one small example before coding.
4. Give the brute-force idea first. State its data structures and complexity.
5. Say exactly why brute force is too slow or too memory-heavy.
6. Identify the pattern for the optimal solution: graph, heap, sliding window, BFS, binary search, etc.
7. Explain the optimal algorithm in steps before touching the keyboard.
8. Code while narrating only meaningful decisions; do not narrate syntax.
9. Dry-run one normal example and one edge case.
10. Finish with time/space complexity and one scalability follow-up.

For every LLD problem

11. Clarify scope. Do not design the whole AWS service unless asked.
12. State the main use cases and what is explicitly out of scope.
13. Name the entities and responsibilities before defining methods.
14. Prefer composition and small interfaces over deep inheritance.
15. Name the design pattern and explain why it earns its place; never force a pattern.
16. Implement the core flow only. Leave persistence/networking as interfaces or follow-ups.
17. Finish with extensibility: new task type, new strategy, concurrency, failures, retries, metrics.

DSA 1: Course Schedule II - Dependency Ordering (LC 210)

Interview prompt: You have numCourses courses and prerequisite pairs [course, prerequisite]. Return any valid order to finish all courses. If no valid order exists, return an empty list.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Does [a, b] mean b must be completed before a?"
- **You:** "Can the input contain duplicate prerequisite pairs?"
- **You:** "If multiple valid orders exist, can I return any one of them?"
- **You:** "If there is a cycle, should I return an empty list?"
- **You:** "What are the approximate numbers of courses and prerequisite edges?"

2. Understanding of the question

This is a dependency problem. Every course is a node. A prerequisite b -> a is a directed edge. We need a linear ordering that respects every edge. Such an ordering exists if and only if the directed graph has no cycle.

3. Brute-force approach

English idea

Repeatedly scan all unfinished courses. If a course has all prerequisites already completed, take it. Keep scanning until every course is taken. If a full pass makes no progress, the remaining courses depend on one another in a cycle.

Data structures

- A prerequisite map: course -> list of prerequisites.
- A set of completed courses.
- A set/list of remaining courses.
- An output list for the order.

Algorithm / how the code works

18. Build the prerequisite list for every course.
19. Put every course in a remaining set.
20. On each pass, inspect every remaining course.
21. If all of its prerequisites are in completed, append the course to the answer and remove it from remaining.
22. If an entire pass adds nothing, return [] because the remaining graph contains a cycle.

Time and space complexity

Measure	Complexity	Why
Time	O(V * (V + E)) worst case	A long dependency chain may require V passes; each pass can scan many courses and prerequisite checks.
Space	O(V + E)	Prerequisite lists, completed set, remaining set, and output.

Fully commented brute-force code

```
from typing import List

class Solution:
    def findOrder(self, numCourses: int,
                  prerequisites: List[List[int]]) -> List[int]:
        # prereq[c] contains every course that must be finished before c.
        prereq = [[] for _ in range(numCourses)]
        for course, before in prerequisites:
            prereq[course].append(before)

        completed = set()
        remaining = set(range(numCourses))
        order = []
```

```
        while remaining:
            made_progress = False

            # list(...) lets us safely remove from remaining while iterating.
            for course in list(remaining):
                # This is the expensive brute-force check.
                if all(before in completed for before in prereq[course]):
                    completed.add(course)
                    remaining.remove(course)
                    order.append(course)
                    made_progress = True

            # No available course means the unfinished courses form a cycle.
            if not made_progress:
                return []

        return order
```

Limitation of brute force

- It repeatedly rechecks the same prerequisite relationships.
- For a long chain, only one course may become available per pass.
- It does not exploit the fact that only neighbors of a completed course can become newly available.

4. Optimal / interview-ready approach

English idea

Use Kahn's topological sort. Count each course's indegree: how many prerequisites are still unsatisfied. Start with every course whose indegree is 0. Whenever we finish one, decrement the indegree of courses that depend on it. If all V courses are processed, the order is valid; otherwise a cycle exists.

Data structures

- Adjacency list: prerequisite -> courses unlocked by it.
- Indegree array.
- Queue of currently available courses.
- Output list.

Algorithm / how the code works

23. Build adjacency list and indegree from every prerequisite edge.
24. Push all indegree-0 courses into a queue.
25. Pop one available course and append it to the order.
26. For each neighbor, decrement indegree because one prerequisite is now satisfied.
27. If a neighbor reaches indegree 0, add it to the queue.
28. At the end, return the order only if its length is numCourses.

Time and space complexity

Measure	Complexity	Why
Time	O(V + E)	Every course enters/leaves the queue once and every edge is processed once.
Space	O(V + E)	Adjacency list, indegree array, queue, and output.

Fully commented optimal code

```
from collections import deque
from typing import List

class Solution:
    def findOrder(self, numCourses: int,
                  prerequisites: List[List[int]]) -> List[int]:
        # graph[before] = courses that become closer to available
        # after "before" is completed.
        graph = [[] for _ in range(numCourses)]
        indegree = [0] * numCourses

        for course, before in prerequisites:
            graph[before].append(course)
```

```
            indegree[course] += 1

        # Courses with no unfinished prerequisites can be taken now.
        q = deque(
            course for course in range(numCourses)
            if indegree[course] == 0
        )

        order = []

        while q:
            course = q.popleft()
            order.append(course)

            # Completing this course satisfies one prerequisite
            # for every dependent course.
            for nxt in graph[course]:
                indegree[nxt] -= 1
                if indegree[nxt] == 0:
                    q.append(nxt)

        # If some course was never processed, a cycle exists.
        return order if len(order) == numCourses else []
```

5. Edge cases to say out loud

- No courses.
- A course with no prerequisites.
- Disconnected dependency groups.
- A direct cycle such as 0 -> 1 -> 0.
- Duplicate edges if the interviewer permits them.

6. Interviewer follow-ups / last-minute notes

- LC 207 Course Schedule is the same graph: return len(order) == numCourses instead of the order.
- Team framing: deployments/jobs with dependencies. Say "topological sort" as soon as you recognize it.

DSA 2: Task Scheduler - Priority + Cooldown (LC 621)

Interview prompt: Given CPU tasks represented by letters and a cooldown n, return the minimum number of time slots needed so identical tasks are separated by at least n slots.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Can I reorder tasks arbitrarily?"
- **You:** "Does an idle slot count toward the answer?"
- **You:** "Are all tasks one time unit long?"
- **You:** "If n is zero, can identical tasks run back-to-back?"

2. Understanding of the question

We want to schedule every task while obeying a cooldown between two executions of the same task. The challenge is to avoid idle time by always choosing a useful eligible task when possible.

3. Brute-force approach

English idea

At every time slot, scan every task type and choose the eligible task with the largest remaining count. Track when each task type is allowed to run again. If no task is eligible, spend an idle slot.

Data structures

- Counter of remaining task frequencies.
- Dictionary task -> next allowed time.
- A time counter.

Algorithm / how the code works

29. Count each task type.
30. At the current time, scan all task types.
31. Among tasks with count > 0 and next_allowed <= time, choose the one with largest remaining count.
32. If one exists, execute it, decrement its count, and set its next allowed time to time + n + 1.
33. If none exists, the slot is idle.
34. Repeat until all task counts become zero.

Time and space complexity

Measure	Complexity	Why
Time	O(T * U)	For each of T schedule slots, scan U distinct task types to select an eligible task.
Space	O(U)	Frequency and next-available maps for U task types.

Fully commented brute-force code

```
from collections import Counter
from typing import List

class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        remaining = Counter(tasks)
        next_allowed = {task: 0 for task in remaining}

        time = 0
        tasks_left = len(tasks)

        while tasks_left > 0:
            best_task = None
            best_count = -1

            # Brute force: scan every distinct task every time slot.
```

```
            for task, count in remaining.items():
                if count == 0:
                    continue
                if next_allowed[task] > time:
                    continue
                if count > best_count:
                    best_count = count
                    best_task = task

            if best_task is not None:
                remaining[best_task] -= 1
                tasks_left -= 1
                next_allowed[best_task] = time + n + 1

            # Time advances even if this was an idle slot.
            time += 1

        return time
```

Limitation of brute force

- Scanning all task types every slot is unnecessary.
- Large cooldowns can create many idle slots, making T larger than the number of tasks.
- The selection operation should use a priority structure rather than a full scan.

4. Optimal / interview-ready approach

English idea

Use a max heap for currently available tasks and a queue for tasks cooling down. The max heap always gives the task with the largest remaining count. After executing a task, put it in the cooldown queue with the first time it can be used again.

Data structures

- Counter for initial frequencies.
- Max heap (implemented with negative counts).
- Deque of (ready_time, remaining_count).

Algorithm / how the code works

35. Count task frequencies and heapify all counts as negative values.
36. At each time, first move every cooled-down task whose ready_time <= time back into the heap.
37. If the heap is non-empty, execute the most frequent available task.
38. If that task still has remaining occurrences, append it to cooldown with ready_time = time + n + 1.
39. Advance time until both heap and cooldown queue are empty.

Time and space complexity

Measure	Complexity	Why
Time	O(T log U)	Each execution performs heap push/pop over at most U task types; T includes unavoidable idle slots.
Space	O(U)	Heap and cooldown queue hold at most the distinct task types.

Fully commented optimal code

```
from collections import Counter, deque
import heapq
from typing import List

class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        # Negative counts turn Python's min heap into a max heap.
        max_heap = [-count for count in Counter(tasks).values()]
        heapq.heapify(max_heap)

        # Each entry is (time_when_task_can_run_again, negative_count).
        cooldown = deque()
        time = 0

        while max_heap or cooldown:
```

```
            # Return every task whose cooldown has finished.
            while cooldown and cooldown[0][0] <= time:
                _, neg_count = cooldown.popleft()
                heapq.heappush(max_heap, neg_count)

            if max_heap:
                neg_count = heapq.heappop(max_heap)

                # Example: -3 -> -2 means two copies remain.
                neg_count += 1

                if neg_count < 0:
                    ready_time = time + n + 1
                    cooldown.append((ready_time, neg_count))

            # If max_heap was empty, this slot is simply idle.
            time += 1

        return time
```

5. Edge cases to say out loud

- n = 0.
- Only one task type.
- All task types have the same frequency.
- A very large cooldown.
- Empty task list if the interviewer allows it.

6. Interviewer follow-ups / last-minute notes

- There is also an O(N) mathematical formula solution. The heap version is easier to adapt when an interviewer adds priorities, durations, or machine availability.
- Fleet framing: jobs wait for eligible execution; heap = priority, queue = temporarily unavailable work.

DSA 3: Top K Frequent Elements (LC 347)

Interview prompt: Given an integer array nums and integer k, return the k most frequent elements.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Is k guaranteed to be between 1 and the number of unique values?"
- **You:** "Can I return the answer in any order?"
- **You:** "If frequencies tie, does the order matter?"
- **You:** "How large can nums be?"

2. Understanding of the question

The first step is unavoidable: count how many times each value occurs. Then we need only the k largest frequencies, not a complete global ordering of all unique values.

3. Brute-force approach

English idea

Count frequencies, convert them to (value, frequency) pairs, sort all unique values by frequency descending, and return the first k.

Data structures

- HashMap/Counter for frequencies.
- Array/list of unique values for sorting.

Algorithm / how the code works

40. Count each value.
41. Sort all unique values using their frequency as the key.
42. Take the first k values.

Time and space complexity

Measure	Complexity	Why
Time	O(N + U log U)	Counting is O(N); sorting all U unique values is O(U log U).
Space	O(U)	Frequency map and sorted unique elements.

Fully commented brute-force code

```
from collections import Counter
from typing import List

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        freq = Counter(nums)

        # Brute force: globally sort every unique value.
        ordered = sorted(
            freq.keys(),
            key=lambda value: freq[value],
            reverse=True
        )

        return ordered[:k]
```

Limitation of brute force

- Sorting all U unique values does extra work when k is small.
- For example, if U = 1,000,000 and k = 10, we only need the best 10, not a complete ranking of all 1,000,000.

4. Optimal / interview-ready approach

English idea

Maintain a min heap of size at most k. For each unique value, push (frequency, value). If the heap grows beyond k, remove the smallest frequency. At the end the heap contains exactly the k most frequent elements.

Data structures

- HashMap/Counter for frequencies.
- Min heap of size k.

Algorithm / how the code works

43. Count every value.
44. Iterate through unique values and push (frequency, value) into the heap.
45. Whenever heap size exceeds k, pop the least frequent candidate.
46. Return the values left in the heap.

Time and space complexity

Measure	Complexity	Why
Time	O(N + U log K)	Counting is O(N); each of U unique values performs a heap operation on size at most K.
Space	O(U + K)	The frequency map stores U entries and the heap stores at most K.

Fully commented optimal code

```
from collections import Counter
import heapq
from typing import List

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        freq = Counter(nums)
        min_heap = []

        for value, count in freq.items():
            # Heap is ordered by frequency first.
            heapq.heappush(min_heap, (count, value))

            # Keep only the best k candidates seen so far.
            if len(min_heap) > k:
                heapq.heappop(min_heap)

        return [value for count, value in min_heap]
```

5. Edge cases to say out loud

- k = 1.
- k equals the number of unique values.
- All values have the same frequency.
- One value dominates the entire input.
- Negative numbers.

6. Interviewer follow-ups / last-minute notes

- If the interviewer explicitly wants O(N), mention bucket sort by frequency. Heap is usually the cleanest answer when k is small or the data is streaming.
- Fleet framing: top K hosts/error types by count.

DSA 4: Rolling-Window Top K Events - Custom Stream Problem

Interview prompt: Events arrive as (timestamp, item). At any time, return the K items that occurred most frequently during only the last W time units.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Are events guaranteed to arrive in nondecreasing timestamp order?"
- **You:** "Is the window [now-W, now] inclusive on both ends?"
- **You:** "Will topK be queried after every event or only occasionally?"
- **You:** "Do ties need deterministic ordering?"
- **You:** "Can the same item appear many times in the window?"

2. Understanding of the question

The key difference from normal Top K is expiration. An event that was important one minute ago may leave the window now. We therefore need to remove old events as time advances, maintain current frequencies, and compute the best k among only live items.

3. Brute-force approach

English idea

Keep every event ever seen. For each query, scan the entire history, count only events whose timestamps are inside the requested window, sort the frequency map, and return the top k.

Data structures

- List of all historical events.
- Temporary Counter built for every query.
- Sorting of unique live items.

Algorithm / how the code works

47. Append every incoming event to history.
48. For topK(now), scan all historical events.
49. If timestamp >= now - window, count the item.
50. Sort live items by frequency descending.
51. Return the first k.

Time and space complexity

Measure	Complexity	Why
Add	O(1)	Just append to history.
Query	O(N + U log U)	Rescan all N historical events, then sort U live unique items.
Space	O(N)	Old expired events are never removed.

Fully commented brute-force code

```
from collections import Counter
import heapq

class BruteRollingTopK:
    def __init__(self, window_size: int):
        self.window_size = window_size
        self.events = []

    def add(self, timestamp: int, item: str) -> None:
        self.events.append((timestamp, item))

    def top_k(self, now: int, k: int):
        freq = Counter()
        left = now - self.window_size
```

```
        # Brute force: revisit every event ever received.
        for timestamp, item in self.events:
            if timestamp >= left:
                freq[item] += 1

        return heapq.nlargest(k, freq.keys(), key=freq.get)
```

Limitation of brute force

- Memory grows forever.
- Every query repeatedly processes events that expired long ago.
- The window state should be maintained incrementally as timestamps advance.

4. Optimal / interview-ready approach

English idea

Use a deque to store only events currently inside the window and a Counter for their frequencies. Before every add/query, pop expired events from the left of the deque and decrement their counts. Then use a size-k selection (`heapq.nlargest`) over only the live unique items.

Data structures

- Deque of live (timestamp, item) events.
- Counter item -> live frequency.
- Heap-based top-k selection at query time.

Algorithm / how the code works

52. Before processing time now, repeatedly pop deque[0] while its timestamp < now - W.
53. For each expired event, decrement its item frequency and delete zero counts.
54. When a new event arrives, append it and increment its frequency.
55. For top_k, run heapq.nlargest(k, live_items, key=current frequency).

Time and space complexity

Measure	Complexity	Why
Add / expire	Amortized O(1) per event	Each event enters the deque once and leaves once.
Query	O(U log K)	Select K winners from U unique live items without sorting all U.
Space	O(M + U)	M is the number of events currently inside the window, not total history.

Fully commented optimal code

```
from collections import Counter, deque
import heapq

class RollingTopK:
    def __init__(self, window_size: int):
        self.window_size = window_size
        self.window = deque()      # Only live events.
        self.freq = Counter()      # Frequency inside current window.

    def _expire(self, now: int) -> None:
        left = now - self.window_size

        # Events are timestamp-ordered, so expired events are at the front.
        while self.window and self.window[0][0] < left:
            timestamp, item = self.window.popleft()
            self.freq[item] -= 1

            if self.freq[item] == 0:
                del self.freq[item]

    def add(self, timestamp: int, item: str) -> None:
        self._expire(timestamp)
        self.window.append((timestamp, item))
```

```
        self.freq[item] += 1

    def top_k(self, now: int, k: int):
        self._expire(now)

        # nlargest internally keeps only k best candidates.
        return heapq.nlargest(
            k,
            self.freq.keys(),
            key=self.freq.get
        )
```

5. Edge cases to say out loud

- No events in the window.
- k > number of unique live items.
- Many events at exactly the expiration boundary.
- Multiple events with the same timestamp.
- Out-of-order timestamps: clarify; if allowed, deque logic needs a different structure.

6. Interviewer follow-ups / last-minute notes

- Do not over-engineer a fully dynamic heap unless the interviewer specifically requires very frequent O(log U) queries. The deque + Counter + nlargest solution is clean, correct, and easy to explain.
- Fleet framing: top K hosts with most failures in the last 10 minutes.

DSA 5: LRU Cache (LC 146)

Interview prompt: Design a fixed-capacity cache supporting get(key) and put(key, value). When full, evict the least recently used key. Target O(1) average time for both operations.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Does get update recency?"
- **You:** "Does updating an existing key update recency?"
- **You:** "Is capacity always positive?"
- **You:** "What should get return for a missing key?"

2. Understanding of the question

We need two capabilities simultaneously: O(1) lookup by key and O(1) movement/removal based on recency. A HashMap gives lookup but not recency order; a doubly linked list gives O(1) node removal/movement when we already have the node.

3. Brute-force approach

English idea

Use a dictionary for values and a Python list for recency. The end of the list is most recent. Every get/put removes the key from its current list position and appends it to the end. Eviction removes index 0.

Data structures

- Dictionary key -> value.
- List of keys ordered least-recent to most-recent.

Algorithm / how the code works

56. On get, return -1 if missing.
57. If present, remove the key from the list and append it to the end.
58. On put, if key exists update value and move it to the end.
59. If new and full, remove the first key from the list and dictionary.
60. Append the new key as most recent.

Time and space complexity

Measure	Complexity	Why
Get	O(C)	list.remove(key) may scan up to cache capacity C.
Put	O(C)	Removing/moving an existing key may scan the list.
Space	O(C)	Dictionary and recency list store at most C keys.

Fully commented brute-force code

```
class BruteLRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.values = {}
        self.order = []  # Least recent at index 0.

    def get(self, key: int) -> int:
        if key not in self.values:
            return -1

        # O(C): finding/removing key from Python list.
        self.order.remove(key)
        self.order.append(key)
        return self.values[key]
```

```
    def put(self, key: int, value: int) -> None:
        if key in self.values:
            self.values[key] = value
            self.order.remove(key)
            self.order.append(key)
            return

        if len(self.values) == self.capacity:
            lru_key = self.order.pop(0)  # Also O(C) due to shifting.
            del self.values[lru_key]

        self.values[key] = value
        self.order.append(key)
```

Limitation of brute force

- List removal and pop(0) are linear.
- It cannot meet the required O(1) get/put target.
- We need direct access to the recency node, not just to the value.

4. Optimal / interview-ready approach

English idea

Use a HashMap key -> linked-list node and a doubly linked list ordered from most recent to least recent. Sentinel head/tail nodes remove edge-case code. Every touched node moves to the front; eviction removes tail.prev.

Data structures

- HashMap key -> Node.
- Doubly linked list.
- Two sentinel nodes: head (MRU side) and tail (LRU side).

Algorithm / how the code works

61. Map gives the node in O(1).
62. Detach a node by changing four pointers.
63. Insert a node immediately after head to mark it most recent.
64. On get, move the found node to front.
65. On put, update/move existing nodes or create a new node.
66. If capacity is exceeded, remove tail.prev and delete its map entry.

Time and space complexity

Measure	Complexity	Why
Get	O(1) average	Hash lookup plus constant pointer changes.
Put	O(1) average	Hash update plus constant pointer changes and constant-time eviction.
Space	O(C)	At most C real nodes plus two sentinels and C map entries.

Fully commented optimal code

```
class Node:
    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.nodes = {}  # key -> Node

        # Sentinels avoid special cases for empty/one-node lists.
        self.head = Node()  # Most-recent side.
        self.tail = Node()  # Least-recent side.
        self.head.next = self.tail
```

```
        self.tail.prev = self.head

    def _remove(self, node: Node) -> None:
        """Detach node from its current list position."""
        node.prev.next = node.next
        node.next.prev = node.prev

    def _add_front(self, node: Node) -> None:
        """Insert node immediately after head (most recent)."""
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def _make_recent(self, node: Node) -> None:
        self._remove(node)
        self._add_front(node)

    def get(self, key: int) -> int:
        if key not in self.nodes:
            return -1

        node = self.nodes[key]
        self._make_recent(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if key in self.nodes:
            node = self.nodes[key]
            node.value = value
            self._make_recent(node)
            return

        node = Node(key, value)
        self.nodes[key] = node
        self._add_front(node)

        if len(self.nodes) > self.capacity:
            # tail.prev is the least recently used real node.
            lru = self.tail.prev
            self._remove(lru)
            del self.nodes[lru.key]
```

5. Edge cases to say out loud

- Capacity 1.
- Get missing key.
- Update existing key.
- Eviction immediately after a get changes which key is LRU.
- Repeated gets of the same key.

6. Interviewer follow-ups / last-minute notes

- Follow-up: TTL adds expiration timestamps; thread safety adds a lock around mutations; a distributed cache adds sharding, replication, invalidation, and consistency questions.

DSA 6: Merge Intervals (LC 56)

Interview prompt: Given intervals [start, end], merge all overlapping intervals and return the non-overlapping result.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Are intervals closed, so [1,4] and [4,5] count as overlapping?"
- **You:** "Can intervals arrive unsorted?"
- **You:** "Can there be duplicate intervals?"
- **You:** "Is start always <= end?"

2. Understanding of the question

Intervals overlap when the next interval begins before or at the current merged interval's end (for closed intervals). Sorting by start time makes all possible overlaps appear next to one another.

3. Brute-force approach

English idea

Repeatedly search every pair of intervals for an overlap. Merge the first overlapping pair you find, replace the pair with the merged interval, then restart until no overlap remains.

Data structures

- Mutable list of intervals.
- Nested loops to find an overlapping pair.

Algorithm / how the code works

67. Copy the intervals.
68. Search all i,j pairs.
69. If two overlap, replace them with their union.
70. Restart the search because the new merged interval may overlap something else.
71. Stop when one full pass finds no overlap.

Time and space complexity

Measure	Complexity	Why
Time	O(N^3) worst case	Up to N merges; each restart can scan O(N^2) pairs.
Space	O(N)	Working list of intervals.

Fully commented brute-force code

```
from typing import List

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        work = [interval[:] for interval in intervals]

        changed = True
        while changed:
            changed = False

            for i in range(len(work)):
                if changed:
                    break

                for j in range(i + 1, len(work)):
                    a, b = work[i], work[j]

                    # Closed intervals overlap unless one is fully before other.
                    overlap = not (a[1] < b[0] or b[1] < a[0])
                    if overlap:
                        merged = [min(a[0], b[0]), max(a[1], b[1])]
```

```
                    # Remove larger index first, then smaller index.
                    work.pop(j)
                    work.pop(i)
                    work.append(merged)

                    changed = True
                    break

        return work
```

Limitation of brute force

- Repeatedly comparing all pairs is extremely wasteful.
- A merge can cause a complete restart.
- Sorting once gives a simple one-pass structure afterward.

4. Optimal / interview-ready approach

English idea

Sort intervals by start time. Keep a result list. If the next interval starts after the last merged interval ends, append it. Otherwise, extend the last merged interval's end.

Data structures

- Sorted array of intervals.
- Output list used as the merge stack.

Algorithm / how the code works

72. Sort by interval start.
73. Initialize result with the first interval.
74. For each next interval, compare its start with result[-1].end.
75. If overlapping, set result[-1].end = max(current end, next end).
76. Otherwise append a new interval.

Time and space complexity

Measure	Complexity	Why
Time	O(N log N)	Sorting dominates; the scan is O(N).
Space	O(N) output	The result can contain all N intervals. Sorting may also use implementation-dependent extra space.

Fully commented optimal code

```
from typing import List

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        if not intervals:
            return []

        intervals.sort(key=lambda interval: interval[0])
        merged = [intervals[0][:]]

        for start, end in intervals[1:]:
            last_end = merged[-1][1]

            if start <= last_end:
                # Current interval overlaps the previous merged interval.
                merged[-1][1] = max(last_end, end)
            else:
                # No overlap; start a new merged interval.
                merged.append([start, end])

        return merged
```

5. Edge cases to say out loud

- Empty input.
- One interval.
- Intervals fully contained inside another.
- Touching endpoints.
- Already sorted/non-overlapping input.

6. Interviewer follow-ups / last-minute notes

- Fleet framing: maintenance windows or deployment windows. If asked for max simultaneous intervals instead of merging, think Meeting Rooms II.

DSA 7: Number of Islands (LC 200)

Interview prompt: Given a grid of 1s (land) and 0s (water), return the number of connected land components using 4-directional adjacency.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Is adjacency only up/down/left/right, not diagonal?"
- **You:** "Can I modify the input grid?"
- **You:** "Can the grid be empty?"
- **You:** "Are cells strings "1"/"0" or integers?"

2. Understanding of the question

Each island is one connected component. A graph traversal from one land cell can discover the entire island. The key optimization is to ensure every cell is processed at most once globally.

3. Brute-force approach

English idea

From every land cell, run a fresh BFS that discovers its entire component, convert that component to a canonical set/tuple, and store it in a set of components. Because there is no global visited set, the same island is rediscovered from every land cell in it.

Data structures

- Queue for each BFS.
- Temporary visited set for one BFS.
- Set of canonical components to deduplicate repeated discoveries.

Algorithm / how the code works

77. For every land cell, start a BFS.
78. Explore all connected land cells and collect their coordinates.
79. Sort/freeze the component and add it to a set.
80. After all land starting points are processed, return the number of unique components.

Time and space complexity

Measure	Complexity	Why
Time	O((R*C)^2) worst case	If the whole grid is land, BFS over R*C cells is repeated from R*C starting cells.
Space	O((R*C)^2) worst case	Storing many repeated component representations is very expensive.

Fully commented brute-force code

```
from collections import deque
from typing import List

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0

        rows, cols = len(grid), len(grid[0])
        components = set()

        def discover(sr, sc):
            q = deque([(sr, sc)])
            seen = {(sr, sc)}

            while q:
                r, c = q.popleft()
```

```
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    nr, nc = r + dr, c + dc
                    if (0 <= nr < rows and 0 <= nc < cols and
                            grid[nr][nc] == "1" and
                            (nr, nc) not in seen):
                        seen.add((nr, nc))
                        q.append((nr, nc))

            # Canonical immutable representation for deduplication.
            return tuple(sorted(seen))

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == "1":
                    components.add(discover(r, c))

        return len(components)
```

Limitation of brute force

- The same island is traversed again from every land cell.
- It wastes both time and memory storing duplicate component descriptions.
- A global visited set (or in-place mutation) eliminates all repeated work.

4. Optimal / interview-ready approach

English idea

Scan the grid once. Whenever you find unvisited land, increment the island count and BFS/DFS from that cell, marking the whole island visited. Every cell is visited at most once.

Data structures

- Queue for BFS (or stack/recursion for DFS).
- Visited set, unless the interviewer allows modifying the grid.

Algorithm / how the code works

81. Scan every cell.
82. If the cell is land and not visited, this is a new island: count += 1.
83. BFS from it and mark every connected land cell visited.
84. Continue scanning; visited land will be skipped.

Time and space complexity

Measure	Complexity	Why
Time	O(R*C)	Every grid cell is examined a constant number of times.
Space	O(R*C) worst case	Visited set and queue can hold a large component. If input mutation is allowed, visited storage can be removed.

Fully commented optimal code

```
from collections import deque
from typing import List

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0

        rows, cols = len(grid), len(grid[0])
        visited = set()
        islands = 0

        def bfs(sr: int, sc: int) -> None:
            q = deque([(sr, sc)])
            visited.add((sr, sc))

            while q:
                r, c = q.popleft()
```

```
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nr, nc = r + dr, c + dc

                if (0 <= nr < rows and 0 <= nc < cols and
                        grid[nr][nc] == "1" and
                        (nr, nc) not in visited):
                    visited.add((nr, nc))
                    q.append((nr, nc))

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1" and (r, c) not in visited:
                islands += 1
                bfs(r, c)

    return islands
```

5. Edge cases to say out loud

- All water.
- All land.
- Single cell.
- Several diagonal land cells (not connected if only 4 directions).
- Long skinny island.

6. Interviewer follow-ups / last-minute notes

- Why BFS vs DFS? Both are O(R*C). BFS uses a queue and avoids recursion-depth risk; DFS is often shorter. The choice is not about correctness here.
- Fleet framing: connected server clusters/failure regions.

DSA 8: Time Based Key-Value Store (LC 981)

Interview prompt: Support set(key, value, timestamp) and get(key, timestamp), which returns the value associated with the greatest stored timestamp <= the requested timestamp.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Are set timestamps strictly increasing for each key?"
- **You:** "What should get return if no timestamp <= query exists?"
- **You:** "Can the same key have many versions?"
- **You:** "Are timestamps integers?"

2. Understanding of the question

Each key has a history of versions. get does not necessarily want the newest version overall; it wants the newest version that existed at or before a requested time.

3. Brute-force approach

English idea

For every key, append (timestamp, value). On get, scan that key's versions from newest to oldest until the first timestamp <= query timestamp is found.

Data structures

- HashMap key -> list of (timestamp, value).

Algorithm / how the code works

85. Append every set to the key's version list.
86. For get, iterate reversed(version_list).
87. Return the first pair with stored_timestamp <= query.
88. If none qualifies, return empty string.

Time and space complexity

Measure	Complexity	Why
Set	O(1)	Append to a list.
Get	O(M)	May scan all M versions for that key.
Space	O(total sets)	Every version must be stored.

Fully commented brute-force code

```
from collections import defaultdict

class TimeMap:
    def __init__(self):
        self.store = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        # Brute force: walk backward through all versions.
        for saved_time, value in reversed(self.store[key]):
            if saved_time <= timestamp:
                return value

        return ""
```

Limitation of brute force

- A key with hundreds of thousands of versions can make get linear.
- Because timestamps are ordered, we should exploit sorted order with binary search.

4. Optimal / interview-ready approach

English idea

Keep the same append-only version list, but use binary search to find the rightmost timestamp <= query. This is the standard "rightmost valid value" binary-search pattern.

Data structures

- HashMap key -> sorted list of (timestamp, value).
- Binary search indices left/right.

Algorithm / how the code works

89. Append on set because timestamps are increasing.
90. On get, binary search the list.
91. Whenever mid.timestamp <= query, save that value and search right for a newer valid version.
92. If mid.timestamp > query, search left.
93. Return the last valid value found.

Time and space complexity

Measure	Complexity	Why
Set	O(1)	Append-only when timestamps arrive in increasing order.
Get	O(log M)	Binary search among M versions for the key.
Space	O(total sets)	All historical versions are retained.

Fully commented optimal code

```
from collections import defaultdict

class TimeMap:
    def __init__(self):
        self.store = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        # Problem guarantee: timestamps increase for each key.
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        versions = self.store[key]
        left, right = 0, len(versions) - 1
        answer = ""

        while left <= right:
            mid = (left + right) // 2
            saved_time, value = versions[mid]

            if saved_time <= timestamp:
                # Valid candidate. Try to find a newer valid version.
                answer = value
                left = mid + 1
            else:
                right = mid - 1

        return answer
```

5. Edge cases to say out loud

- Missing key.
- Query earlier than first version.
- Query exactly equals a version timestamp.
- Query later than latest version.
- Many versions for one key.

6. Interviewer follow-ups / last-minute notes

- Fleet framing: retrieve a host configuration/state as of time T. This is a natural infrastructure variation.

DSA 9: Rotting Oranges (LC 994)

Interview prompt: In a grid, 0 is empty, 1 is fresh orange, and 2 is rotten. Every minute, fresh oranges adjacent to rotten ones become rotten. Return the minimum minutes until no fresh orange remains, or -1 if impossible.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "Is adjacency four-directional only?"
- **You:** "Do all currently rotten oranges spread simultaneously each minute?"
- **You:** "If there are no fresh oranges initially, should I return 0?"
- **You:** "Can I modify the grid?"

2. Understanding of the question

This is shortest-time propagation from multiple starting points. Every initially rotten orange is a source at minute 0. All sources spread simultaneously, which is exactly multi-source BFS.

3. Brute-force approach

English idea

Simulate minute by minute. During every minute, scan the entire grid to find fresh oranges adjacent to any rotten orange. Mark those positions, then rot them all simultaneously after the scan.

Data structures

- Grid.
- Temporary list/set of oranges that rot this minute.
- Fresh-orange counter.

Algorithm / how the code works

94. Count fresh oranges.
95. For each minute, scan every cell.
96. If a fresh orange has a rotten neighbor, add it to to_rot.
97. After the full scan, rot every cell in to_rot.
98. If no cell rots while fresh oranges remain, return -1.
99. Otherwise repeat and count minutes.

Time and space complexity

Measure	Complexity	Why
Time	O(T * R * C)	Each of T minutes scans the full grid; T can be O(R*C).
Space	O(R*C)	Temporary set/list of oranges rotting during a minute.

Fully commented brute-force code

```
from typing import List

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        fresh = sum(cell == 1 for row in grid for cell in row)
        minutes = 0

        directions = ((1, 0), (-1, 0), (0, 1), (0, -1))

        while fresh > 0:
            to_rot = set()

            # Brute force: scan the complete grid every minute.
            for r in range(rows):
                for c in range(cols):
```

```
                if grid[r][c] != 1:
                    continue

                for dr, dc in directions:
                    nr, nc = r + dr, c + dc
                    if (0 <= nr < rows and 0 <= nc < cols and
                            grid[nr][nc] == 2):
                        to_rot.add((r, c))
                        break

            if not to_rot:
                return -1

            for r, c in to_rot:
                grid[r][c] = 2
                fresh -= 1

            minutes += 1

        return minutes
```

Limitation of brute force

- Most cells are rescanned every minute even when nothing near them changes.
- The frontier of newly rotten oranges contains exactly the cells that matter next.
- Multi-source BFS processes each cell once instead of rescanning the whole grid.

4. Optimal / interview-ready approach

English idea

Put every initially rotten orange in one queue and count fresh oranges. Process BFS level by level. Each level represents one minute. When a rotten orange reaches a fresh neighbor, mark it rotten immediately, decrement fresh, and enqueue it for the next minute.

Data structures

- Queue initialized with all rotten cells.
- Fresh-orange counter.
- Four direction offsets.

Algorithm / how the code works

100. Scan once: enqueue every rotten orange and count fresh ones.
101. If fresh == 0, return 0.
102. While queue and fresh remain, process exactly the current queue length: one BFS layer.
103. Rot fresh neighbors, decrement fresh, enqueue them.
104. After processing a layer, increment minutes.
105. Return minutes if fresh == 0 else -1.

Time and space complexity

Measure	Complexity	Why
Time	O(R*C)	Each cell is scanned initially and each orange enters the queue at most once.
Space	O(R*C)	Queue can hold many rotten/frontier cells.

Fully commented optimal code

```
from collections import deque
from typing import List

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        q = deque()
        fresh = 0

        # Multi-source BFS starts with every rotten orange.
        for r in range(rows):
            for c in range(cols):
```

```
                if grid[r][c] == 2:
                    q.append((r, c))
                elif grid[r][c] == 1:
                    fresh += 1

        if fresh == 0:
            return 0

        minutes = 0
        directions = ((1, 0), (-1, 0), (0, 1), (0, -1))

        while q and fresh > 0:
            # All nodes currently in q belong to this minute's frontier.
            for _ in range(len(q)):
                r, c = q.popleft()

                for dr, dc in directions:
                    nr, nc = r + dr, c + dc
                    if (0 <= nr < rows and 0 <= nc < cols and
                            grid[nr][nc] == 1):
                        grid[nr][nc] = 2
                        fresh -= 1
                        q.append((nr, nc))

            minutes += 1

        return minutes if fresh == 0 else -1
```

5. Edge cases to say out loud

- No fresh oranges.
- No rotten oranges but fresh oranges exist.
- Fresh orange isolated by empty cells.
- Single row/column.
- Multiple initial rotten sources.

6. Interviewer follow-ups / last-minute notes

- Why BFS instead of DFS? We need minimum time / simultaneous wave distance. BFS levels directly represent minutes. This is the key explanation.

DSA 10: Meeting Rooms II (LC 253)

Interview prompt: Given meeting intervals [start, end], return the minimum number of rooms required so every meeting can occur without overlap.

1. Clarification communication

Say these naturally before coding. You do not need to ask all of them if the prompt already answers one.

- **You:** "If one meeting ends exactly when another starts, can they use the same room?"
- **You:** "Are intervals unsorted?"
- **You:** "Can durations be zero?"
- **You:** "Do I only need the room count, not an assignment?"

2. Understanding of the question

The answer is the maximum number of meetings active at the same time. A meeting ending at time t can free a room for another meeting starting at t.

3. Brute-force approach

English idea

For every meeting start time, count how many intervals are active at that exact time. The maximum count across all starts is the number of rooms needed.

Data structures

- Intervals list.
- Two nested loops / counters.

Algorithm / how the code works

106. For each interval start time t, set active = 0.
107. Scan every interval [s,e].
108. If s <= t < e, that meeting occupies a room at t.
109. Track the maximum active count.

Time and space complexity

Measure	Complexity	Why
Time	O(N^2)	For N start times, scan all N meetings.
Space	O(1)	Only counters beyond the input.

Fully commented brute-force code

```
from typing import List

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        best = 0

        for start_time, _ in intervals:
            active = 0

            for start, end in intervals:
                # End is exclusive: a meeting ending at t frees the room at t.
                if start <= start_time < end:
                    active += 1

            best = max(best, active)

        return best
```

Limitation of brute force

- It repeats the same overlap checks many times.
- We only need to process the chronological start/end events once after sorting.

4. Optimal / interview-ready approach

English idea

Sort all start times and all end times separately. Use two pointers. If the next start is before the next end, a new room is needed. Otherwise, a meeting has ended first, so free a room and advance the end pointer.

Data structures

- Sorted starts array.
- Sorted ends array.
- Two pointers.
- Current rooms and maximum rooms counters.

Algorithm / how the code works

110. Extract and sort start times and end times.
111. Compare starts[i] with ends[j].
112. If start < end, allocate a room and advance i.
113. Else a meeting ended; release a room and advance j.
114. Track the maximum rooms used.

Time and space complexity

Measure	Complexity	Why
Time	O(N log N)	Sorting starts and ends dominates; the pointer scan is O(N).
Space	O(N)	Separate starts and ends arrays.

Fully commented optimal code

```
from typing import List

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        if not intervals:
            return 0

        starts = sorted(start for start, end in intervals)
        ends = sorted(end for start, end in intervals)

        i = j = 0
        rooms_in_use = 0
        max_rooms = 0

        while i < len(starts):
            if starts[i] < ends[j]:
                # Next event is a meeting starting before any room frees.
                rooms_in_use += 1
                max_rooms = max(max_rooms, rooms_in_use)
                i += 1
            else:
                # A meeting ended first (or exactly at the next start).
                rooms_in_use -= 1
                j += 1

        return max_rooms
```

5. Edge cases to say out loud

- Empty intervals.
- One meeting.
- Back-to-back meetings.
- All meetings fully overlap.
- Nested intervals.

6. Interviewer follow-ups / last-minute notes

- Alternative optimal solution: sort intervals by start and maintain a min heap of end times. Use it if the interviewer asks you to assign actual rooms.
- Fleet framing: maximum concurrent maintenance/deployment capacity.

LLD 1: Task / Job Scheduler

Likely prompt: Design a system that accepts jobs, schedules higher-priority jobs first, tracks status, and can support retries for failures.

1. Clarification communication

- **You:** "Are jobs executed immediately when a worker is available, or can they also have a scheduled start time?"
- **You:** "Do jobs have priority? If priorities tie, should earlier submissions win?"
- **You:** "Should failed jobs be retried? If yes, is retry policy configurable?"
- **You:** "Do we need persistence/distributed workers, or should I focus on in-memory LLD classes?"
- **You:** "Do we need cancellation and status lookup?"

Finalized scope / assumptions

- Implement in-memory LLD; persistence and distributed coordination are follow-ups.
- Jobs have id, priority, status, and execute().
- Higher priority runs first; FIFO breaks ties.
- Failed jobs can be retried according to a retry policy.
- Support submit(), run_next(), and get_status().

2. Main entities

Entity	Purpose
Task	Represents one unit of work and its current state.
TaskStatus	PENDING, RUNNING, SUCCEEDED, FAILED.
RetryPolicy	Defines whether a failed task should be attempted again.
Scheduler	Owns the priority queue, chooses the next task, and updates state.
Worker (optional)	Executes a task; can be separated when interviewer asks about multiple workers.

3. Functions of those entities

Entity	Key functions
Task	execute(), id, priority, attempts, status
RetryPolicy	should_retry(task)
Scheduler	submit(task), run_next(), get_status(task_id)
Worker	run(task)

4. Best design pattern and logic

Primary pattern: Strategy + State. Strategy is ideal for retry/scheduling rules because those rules change independently from Task and Scheduler. State is represented explicitly with TaskStatus so legal lifecycle transitions are easy to reason about. The core scheduler stays closed to modification when a new retry policy is added.

Class / interaction flow

```
Client -> Scheduler.submit(Task)
                |
                v
          Priority Queue
                |
                v
          Scheduler.run_next()
                |
                v
             Task.execute()
```

```
            /          \
        success        failure
           |              |
      SUCCEEDED      RetryPolicy?
                          |
                    requeue / FAILED
```

5. Simple code implementation

The goal of this code is interview clarity: clean responsibilities, easy extension, and core behavior. It intentionally avoids production infrastructure details.

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass
from enum import Enum
import heapq


class TaskStatus(Enum):
    PENDING = "PENDING"
    RUNNING = "RUNNING"
    SUCCEEDED = "SUCCEEDED"
    FAILED = "FAILED"


class Task(ABC):
    def __init__(self, task_id: str, priority: int):
        self.task_id = task_id
        self.priority = priority
        self.status = TaskStatus.PENDING
        self.attempts = 0

    @abstractmethod
    def execute(self) -> None:
        """Raise an exception when execution fails."""
        pass


class RetryPolicy(ABC):
    @abstractmethod
    def should_retry(self, task: Task) -> bool:
        pass


class MaxAttemptsRetry(RetryPolicy):
    def __init__(self, max_attempts: int):
        self.max_attempts = max_attempts

    def should_retry(self, task: Task) -> bool:
        return task.attempts < self.max_attempts


class Scheduler:
    def __init__(self, retry_policy: RetryPolicy):
        self.retry_policy = retry_policy
        self.tasks = {}          # task_id -> Task
        self.heap = []           # (-priority, sequence, task_id)
        self.sequence = 0        # FIFO tie-breaker

    def submit(self, task: Task) -> None:
        self.tasks[task.task_id] = task
        heapq.heappush(
            self.heap,
            (-task.priority, self.sequence, task.task_id)
        )
        self.sequence += 1

    def get_status(self, task_id: str) -> TaskStatus:
        return self.tasks[task_id].status

    def run_next(self) -> None:
        if not self.heap:
            return

        _, _, task_id = heapq.heappop(self.heap)
        task = self.tasks[task_id]

        task.status = TaskStatus.RUNNING
        task.attempts += 1

        try:
            task.execute()
```

```
            task.status = TaskStatus.SUCCEEDED
        except Exception:
            if self.retry_policy.should_retry(task):
                task.status = TaskStatus.PENDING
                heapq.heappush(
                    self.heap,
                    (-task.priority, self.sequence, task.task_id)
                )
                self.sequence += 1
            else:
                task.status = TaskStatus.FAILED
```

6. Follow-ups to be ready for

- How do you prevent starvation? Add aging to effective priority.
- What if a worker dies after finishing but before acknowledgment? Use at-least-once delivery + idempotent task execution.
- What if two workers grab the same job? Use an atomic state transition or lease.
- How do delayed retries work? Add ready_time and a delay queue.
- How do dependencies work? Add dependency IDs and only enqueue when all are completed.
- How do you scale? Persistent job store + queue + multiple workers + visibility timeout + DLQ + metrics.

7. Common mistakes to avoid

- Putting every responsibility inside Scheduler.
- Using inheritance just to represent every possible task variation.
- Ignoring task state transitions and duplicate execution.
- Trying to build Kafka/Redis/database code during an LLD round unless asked.

LLD 2: Fleet Deployment Manager

Likely prompt: Design a component that rolls out software/firmware to many hosts safely, supports canary or rolling batches, checks health, and stops or rolls back on failure.

1. Clarification communication

- **You:** "Should I focus on deployment orchestration logic rather than the actual SSH/agent transport?"
- **You:** "Do we support multiple rollout strategies such as canary and rolling?"
- **You:** "What determines whether a batch is healthy enough to continue?"
- **You:** "Should a failed batch automatically stop the deployment?"
- **You:** "Do we need rollback in the core implementation or can I expose it as a method/follow-up?"

Finalized scope / assumptions

- Hosts have id and status.
- A Deployment targets a list of hosts and has a lifecycle state.
- DeploymentStrategy chooses batches.
- HealthChecker decides whether a completed batch is safe.
- DeploymentManager coordinates batch -> deploy -> health check -> continue/stop.

2. Main entities

Entity	Purpose
Host	A machine receiving the new version.
Deployment	Version, target hosts, and deployment status.
DeploymentStrategy	Splits hosts into safe batches.
CanaryStrategy / RollingStrategy	Concrete batching algorithms.
HealthChecker	Evaluates whether a batch is healthy.
DeploymentManager	Orchestrates the rollout lifecycle.

3. Functions of those entities

Entity	Key functions
Host	install(version), rollback()
DeploymentStrategy	batches(hosts)
HealthChecker	is_healthy(hosts)
DeploymentManager	deploy(deployment), rollback(deployment)

4. Best design pattern and logic

Primary pattern: Strategy + State. Strategy is the cleanest fit because rollout algorithms vary independently from orchestration: canary, fixed-size rolling, percentage-based, region-by-region. Deployment state makes STOPPED/SUCCEEDED/RUNNING explicit. The manager depends on abstractions rather than hardcoding one rollout policy.

Class / interaction flow

```
DeploymentManager
    |
    +--> DeploymentStrategy
    |       +--> CanaryStrategy
    |       +--> RollingStrategy
    |
    +--> HealthChecker
    |
```

```
    +--> Deployment
              |
              +--> Hosts

For each batch:
install -> health check -> pass? -> next batch
                        -> fail? -> STOP / rollback
```

5. Simple code implementation

The goal of this code is interview clarity: clean responsibilities, easy extension, and core behavior. It intentionally avoids production infrastructure details.

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from enum import Enum
from typing import List


class DeploymentStatus(Enum):
    PENDING = "PENDING"
    RUNNING = "RUNNING"
    SUCCEEDED = "SUCCEEDED"
    STOPPED = "STOPPED"


@dataclass
class Host:
    host_id: str
    version: str

    def install(self, new_version: str) -> None:
        # Real implementation would call an agent/service.
        self.version = new_version


@dataclass
class Deployment:
    deployment_id: str
    version: str
    hosts: List[Host]
    status: DeploymentStatus = DeploymentStatus.PENDING


class DeploymentStrategy(ABC):
    @abstractmethod
    def batches(self, hosts: List[Host]) -> List[List[Host]]:
        pass


class RollingStrategy(DeploymentStrategy):
    def __init__(self, batch_size: int):
        self.batch_size = batch_size

    def batches(self, hosts: List[Host]) -> List[List[Host]]:
        return [
            hosts[i:i + self.batch_size]
            for i in range(0, len(hosts), self.batch_size)
        ]


class CanaryStrategy(DeploymentStrategy):
    def __init__(self, canary_size: int, later_batch_size: int):
        self.canary_size = canary_size
        self.later_batch_size = later_batch_size

    def batches(self, hosts: List[Host]) -> List[List[Host]]:
        if not hosts:
            return []

        result = [hosts[:self.canary_size]]
        rest = hosts[self.canary_size:]
        for i in range(0, len(rest), self.later_batch_size):
            result.append(rest[i:i + self.later_batch_size])
        return result


class HealthChecker:
    def is_healthy(self, hosts: List[Host]) -> bool:
        # Placeholder: in production check error/latency/heartbeat metrics.
        return True
```

```
class DeploymentManager:
    def __init__(self, strategy: DeploymentStrategy,
                 health_checker: HealthChecker):
        self.strategy = strategy
        self.health_checker = health_checker

    def deploy(self, deployment: Deployment) -> bool:
        deployment.status = DeploymentStatus.RUNNING

        for batch in self.strategy.batches(deployment.hosts):
            for host in batch:
                host.install(deployment.version)

            # Safety gate: never continue blindly after a bad batch.
            if not self.health_checker.is_healthy(batch):
                deployment.status = DeploymentStatus.STOPPED
                return False

        deployment.status = DeploymentStatus.SUCCEEDED
        return True
```

6. Follow-ups to be ready for

- How would rollback work? Store previous version and run rollback in reverse/affected batches.
- How do you avoid deploying an entire rack/AZ at once? Make strategy topology-aware.
- How do retries work? Per-host retry with max attempts and exponential backoff.
- How do you resume after orchestrator crash? Persist deployment/batch/host states.
- How do you ensure a host is not updated twice? Idempotency key = deployment_id + host_id.
- What metrics stop rollout? Error rate, heartbeat loss, latency, install failure %, health alarms.

7. Common mistakes to avoid

- Hardcoding one batch size inside DeploymentManager.
- Deploying all hosts before doing any health check.
- Ignoring partial failure and resumability.
- Confusing LLD orchestration with a full AWS production architecture.

LLD 3: Host Reservation System

Likely prompt: Design a service that lets an internal client reserve an available host for a limited duration, release it, extend the reservation, and prevent double booking.

1. Clarification communication

- **You:** "Can one host have at most one active reservation at a time?"
- **You:** "Do reservations expire automatically after a duration?"
- **You:** "Should clients request a specific host or just a host matching criteria such as region/type?"
- **You:** "For the LLD, can I model atomicity with a lock and then explain a distributed conditional write as the production follow-up?"
- **You:** "Do we need extend and release operations?"

Finalized scope / assumptions

- Each host is AVAILABLE or RESERVED.
- An active reservation has id, owner, host, and expiry time.
- Allocation strategy chooses an available host.
- reserve() must atomically check-and-mark so two clients cannot win the same host.
- Expired reservations should make hosts available again.

2. Main entities

Entity	Purpose
Host	Machine with id, region/type, and reservation state.
Reservation	Lease-like ownership record with expiry time.
HostSelectionStrategy	Chooses which available host to allocate.
ReservationService	Coordinates reserve/release/extend and expiry cleanup.

3. Functions of those entities

Entity	Key functions
HostSelectionStrategy	choose(hosts)
ReservationService	reserve(owner, duration), release(id), extend(id, extra), get(id), cleanup_expired()
Reservation	is_expired(now)

4. Best design pattern and logic

Primary pattern: Strategy + atomic check-and-set (lease semantics). Strategy keeps host-selection policy replaceable (first available, region-aware, least-loaded). The critical correctness rule is not a design pattern: reservation must be atomic. In one process a lock is enough; in a distributed service use a transactional/conditional database update or compare-and-swap with a lease expiry.

Class / interaction flow

```
Client A ----\
              ReservationService -- atomic check/set --> Host
Client B ----/                          |
                                         +--> only one can transition
                                              AVAILABLE -> RESERVED

HostSelectionStrategy decides WHICH host.
Atomic reservation decides WHO wins.
```

5. Simple code implementation

The goal of this code is interview clarity: clean responsibilities, easy extension, and core behavior. It intentionally avoids production infrastructure details.

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from datetime import datetime, timedelta
from enum import Enum
import threading
import uuid


class HostStatus(Enum):
    AVAILABLE = "AVAILABLE"
    RESERVED = "RESERVED"


@dataclass
class Host:
    host_id: str
    region: str
    status: HostStatus = HostStatus.AVAILABLE


@dataclass
class Reservation:
    reservation_id: str
    host_id: str
    owner: str
    expires_at: datetime

    def is_expired(self) -> bool:
        return datetime.utcnow() >= self.expires_at


class HostSelectionStrategy(ABC):
    @abstractmethod
    def choose(self, hosts):
        pass


class FirstAvailableStrategy(HostSelectionStrategy):
    def choose(self, hosts):
        for host in hosts:
            if host.status == HostStatus.AVAILABLE:
                return host
        return None


class ReservationService:
    def __init__(self, hosts, strategy: HostSelectionStrategy):
        self.hosts = {host.host_id: host for host in hosts}
        self.strategy = strategy
        self.reservations = {}
        self.lock = threading.Lock()

    def _cleanup_expired(self) -> None:
        expired_ids = [
            rid for rid, r in self.reservations.items()
            if r.is_expired()
        ]

        for rid in expired_ids:
            reservation = self.reservations.pop(rid)
            self.hosts[reservation.host_id].status = HostStatus.AVAILABLE

    def reserve(self, owner: str, duration_seconds: int):
        # In one process, the lock makes check + update atomic.
        with self.lock:
            self._cleanup_expired()
            host = self.strategy.choose(self.hosts.values())

            if host is None:
                return None

            host.status = HostStatus.RESERVED
            rid = str(uuid.uuid4())
            reservation = Reservation(
                reservation_id=rid,
                host_id=host.host_id,
                owner=owner,
```

```
                expires_at=datetime.utcnow() + timedelta(
                    seconds=duration_seconds
                )
            )
            self.reservations[rid] = reservation
            return reservation

    def release(self, reservation_id: str) -> bool:
        with self.lock:
            reservation = self.reservations.pop(reservation_id, None)
            if reservation is None:
                return False

            self.hosts[reservation.host_id].status = HostStatus.AVAILABLE
            return True

    def extend(self, reservation_id: str, extra_seconds: int) -> bool:
        with self.lock:
            reservation = self.reservations.get(reservation_id)
            if reservation is None or reservation.is_expired():
                return False

            reservation.expires_at += timedelta(seconds=extra_seconds)
            return True
```

6. Follow-ups to be ready for

- Two service instances cannot coordinate with a Python lock. Production answer: DB conditional write/transaction/lease.
- What if a client dies? Lease expires automatically.
- How do you handle heartbeats? Renew/extend the lease while the client is alive.
- How do you reserve by region/type? Add filters to strategy.
- What if the same client retries reserve after timeout? Use an idempotency key.
- How do you scale lookup? Index available hosts by region/type/status.

7. Common mistakes to avoid

- Saying "use a lock" without distinguishing single-process vs distributed atomicity.
- Not giving reservations an expiry time.
- Letting selection and reservation happen in separate non-atomic operations.
- Making Reservation responsible for choosing hosts.

LLD 4: Fleet Monitoring & Alerting System

Likely prompt: Design an LLD component that receives host heartbeats/metrics, evaluates health rules, creates alerts, and notifies one or more channels.

1. Clarification communication

- **You:** "What signals should I support in the core design: heartbeat plus numeric metrics?"
- **You:** "Should rules be configurable and independently addable?"
- **You:** "Can multiple notification channels subscribe to an alert?"
- **You:** "Do we need alert deduplication/suppression in the core implementation or as a follow-up?"
- **You:** "Should I focus on class design rather than streaming infrastructure?"

Finalized scope / assumptions

- Events identify host, metric name/value, and timestamp.
- AlertRule decides whether an event/host state should trigger an alert.
- MonitoringService evaluates all rules.
- AlertChannel sends an alert to email/pager/etc.
- New rules and channels should be addable without changing MonitoringService.

2. Main entities

Entity	Purpose
MetricEvent	One heartbeat or numeric metric sample.
Alert	A detected health problem.
AlertRule	Encapsulates one health rule.
ThresholdRule	Concrete rule for metric threshold.
AlertChannel	Observer interface for delivering alerts.
MonitoringService	Ingests events, evaluates rules, publishes alerts.

3. Functions of those entities

Entity	Key functions
AlertRule	evaluate(event) -> Alert | None
AlertChannel	send(alert)
MonitoringService	ingest(event), add_rule(rule), subscribe(channel)

4. Best design pattern and logic

Primary pattern: Observer + Strategy. Each AlertRule is a Strategy for deciding whether a condition is unhealthy. Notification channels are Observers/subscribers: the monitoring service publishes an alert without knowing channel-specific details. This keeps both rule growth and channel growth clean.

Class / interaction flow

```
MetricEvent
    |
    v
MonitoringService
    |
    +--> AlertRule A
    +--> AlertRule B
    +--> AlertRule C
             |
          Alert?
             |
```

```
              +--> EmailChannel
              +--> PagerChannel
              +--> SlackChannel
```

5. Simple code implementation

The goal of this code is interview clarity: clean responsibilities, easy extension, and core behavior. It intentionally avoids production infrastructure details.

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional


@dataclass
class MetricEvent:
    host_id: str
    metric: str
    value: float
    timestamp: int


@dataclass
class Alert:
    host_id: str
    message: str
    severity: str


class AlertRule(ABC):
    @abstractmethod
    def evaluate(self, event: MetricEvent) -> Optional[Alert]:
        pass


class ThresholdRule(AlertRule):
    def __init__(self, metric: str, max_value: float,
                 severity: str = "HIGH"):
        self.metric = metric
        self.max_value = max_value
        self.severity = severity

    def evaluate(self, event: MetricEvent) -> Optional[Alert]:
        if event.metric == self.metric and event.value > self.max_value:
            return Alert(
                host_id=event.host_id,
                message=f"{event.metric}={event.value} exceeded {self.max_value}",
                severity=self.severity
            )
        return None


class AlertChannel(ABC):
    @abstractmethod
    def send(self, alert: Alert) -> None:
        pass


class ConsolePager(AlertChannel):
    def send(self, alert: Alert) -> None:
        print(f"PAGER [{alert.severity}] {alert.host_id}: {alert.message}")


class MonitoringService:
    def __init__(self):
        self.rules = []
        self.channels = []

    def add_rule(self, rule: AlertRule) -> None:
        self.rules.append(rule)

    def subscribe(self, channel: AlertChannel) -> None:
        self.channels.append(channel)

    def ingest(self, event: MetricEvent) -> None:
        for rule in self.rules:
            alert = rule.evaluate(event)
            if alert is None:
                continue

            for channel in self.channels:
                channel.send(alert)
```

6. Follow-ups to be ready for

- How do you detect missing heartbeats? Store last_seen[host] and run a periodic rule/evaluator.
- How do you avoid duplicate pages every second? Alert state + cooldown/dedup key.
- How do you detect rack-wide incidents? Aggregate by rack/region before alerting.
- What if notification delivery fails? Retry + DLQ; sending should be idempotent if possible.
- How do you scale? Ingestion queue/stream, partition by host, aggregators, durable metrics store, alert service.
- How do you handle rolling windows? Maintain time-bucket counters or windowed stream state.

7. Common mistakes to avoid

- Putting threshold logic directly inside MonitoringService.
- Coupling alert detection to one notification channel.
- Ignoring alert deduplication as a follow-up.
- Using Observer when there is only one fixed channel without explaining why extension is expected.

LLD 5: Distributed Cache / LRU + TTL

Likely prompt: Design a cache for host metadata/configuration with get/put, TTL expiration, and LRU eviction when capacity is reached. Explain how the local component extends to a distributed cache.

1. Clarification communication

- **You:** "Should get return None on a miss?"
- **You:** "Does every key have its own TTL?"
- **You:** "Should a get refresh LRU recency but not TTL?"
- **You:** "Do I need to implement the local cache only and discuss distribution as a follow-up?"
- **You:** "What is the capacity unit: number of entries for this LLD?"

Finalized scope / assumptions

- Local implementation has fixed entry capacity.
- Each entry stores value and absolute expiry time.
- Expired entries behave like misses.
- Any successful get makes the key most recently used.
- When full, evict least recently used entry.
- Distributed concerns are discussed after the core LLD.

2. Main entities

Entity	Purpose
CacheEntry	Value plus expiration metadata.
LRUTTLCache	Core local cache implementing get/put/delete and eviction.
BackingStore (conceptual)	Source of truth used in cache-aside architecture.
CacheService (conceptual)	Loads from source on miss and updates/invalidates cache.

3. Functions of those entities

Entity	Key functions
LRUTTLCache	get(key), put(key,value,ttl), delete(key), _remove_expired_key(key)
BackingStore	load(key), save(key,value)
CacheService	get(key) using cache-aside flow

4. Best design pattern and logic

Primary pattern: Cache-Aside + LRU eviction policy. Cache-Aside is the operational pattern: application checks cache, falls back to source of truth, then fills cache. LRU is the eviction policy. For the interview implementation, Python OrderedDict keeps the code small while preserving the essential O(1) recency operations. If the interviewer forbids it, implement the same policy with HashMap + doubly linked list.

Class / interaction flow

```
Client -> CacheService.get(key)
            |
            +--> Cache hit + not expired -> return
            |
            +--> miss/expired
                   |
                   v
              BackingStore
                   |
                   v
              cache.put(...)
                   |
                 return
```

```
Capacity full -> evict LRU
TTL expired   -> treat as miss
```

5. Simple code implementation

The goal of this code is interview clarity: clean responsibilities, easy extension, and core behavior. It intentionally avoids production infrastructure details.

```
from collections import OrderedDict
from dataclasses import dataclass
import time


@dataclass
class CacheEntry:
    value: object
    expires_at: float


class LRUTTLCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.data = OrderedDict()  # key -> CacheEntry

    def get(self, key):
        entry = self.data.get(key)
        if entry is None:
            return None

        # Expired entries behave exactly like cache misses.
        if time.time() >= entry.expires_at:
            del self.data[key]
            return None

        # Successful read makes the key most recently used.
        self.data.move_to_end(key)
        return entry.value

    def put(self, key, value, ttl_seconds: float) -> None:
        expires_at = time.time() + ttl_seconds

        if key in self.data:
            del self.data[key]

        self.data[key] = CacheEntry(value, expires_at)
        self.data.move_to_end(key)

        # Left side is the least recently used key.
        while len(self.data) > self.capacity:
            self.data.popitem(last=False)

    def delete(self, key) -> None:
        self.data.pop(key, None)


class CacheService:
    def __init__(self, cache: LRUTTLCache, backing_store):
        self.cache = cache
        self.backing_store = backing_store

    def get(self, key):
        value = self.cache.get(key)
        if value is not None:
            return value

        # Cache-aside: load source of truth only on miss.
        value = self.backing_store.load(key)
        if value is not None:
            self.cache.put(key, value, ttl_seconds=60)
        return value
```

6. Follow-ups to be ready for

- How do you make this distributed? Consistent hashing/sharding, replicas, node membership, replication strategy.
- How do you invalidate when source data changes? Delete/update cache on writes, versioning, or short TTL depending consistency needs.
- What is a cache stampede/thundering herd? Many clients miss same key simultaneously; use request coalescing/single-flight or jittered TTL.

- What if cache nodes disagree? Define consistency model; often stale reads are acceptable for metadata but not every safety decision.
- What if 10,000 keys expire at once? Add TTL jitter.
- How do you make local implementation thread-safe? Lock around compound mutations or use a concurrent structure.

7. Common mistakes to avoid

- Treating cache as source of truth.
- Forgetting that successful get changes LRU recency.
- Leaving expired entries visible.
- Saying "distributed cache = Redis" without discussing sharding, replication, invalidation, and failure behavior.

Last-Minute Pattern Recognition Sheet

If the prompt says...	Think...	Core structure
dependencies / prerequisites / valid order	Topological sort	Adjacency list + indegree + queue
top K / most frequent / highest priority	Heap	Counter + size-K min heap
last X minutes / rolling window	Sliding window	Deque + frequency map
least recently used	LRU	HashMap + doubly linked list
merge time ranges	Intervals	Sort by start + scan
connected regions / clusters	BFS/DFS	Visited + queue/stack
minimum minutes for spreading	Multi-source BFS	Queue seeded with all sources
value as of timestamp	Binary search	Key -> sorted versions
max simultaneous intervals	Sweep/two pointers	Sorted starts + ends
safe rollout across machines	Strategy + state	Batches + health gate + rollback
reserve one resource safely	Atomic state transition / lease	Selection strategy + conditional write
many alert rules/channels	Strategy + Observer	Rule interface + subscribers

Complexity Cheat Sheet

Operation / structure	Typical cost	Remember
HashMap get/put	O(1) average	Worst-case theoretical can degrade, but interview answer uses average.
Heap push/pop	O(log N)	Heap top/peek is O(1).
Heapify	O(N)	Not O(N log N).
Sort	O(N log N)	Often acceptable first step for intervals.
BFS/DFS graph	O(V + E)	Each vertex + edge processed once.
BFS/DFS grid	O(R*C)	Each cell processed once.
Binary search	O(log N)	Needs sorted/monotonic structure.
Deque append/popleft	O(1)	Perfect for BFS and rolling windows.
List remove / pop(0)	O(N)	Main reason brute LRU fails.
Doubly linked node remove	O(1)	Only if you already have the node pointer.

Fleet / Distributed-Systems Follow-Up Cheat Sheet

Concept	30-second interview answer
Idempotency	Repeating the same operation produces the same final effect. Critical when retries can execute a job more than once. Use an idempotency key or conditional state transition.
At-least-once delivery	A job may be delivered more than once but should not be lost. Requires idempotent consumers/deduplication.
At-most-once delivery	A job is processed zero or one time. Avoids duplicates but failures can lose work.
Retry + exponential backoff	Retry transient failures with increasing delays to avoid hammering an unhealthy dependency.
Jitter	Randomness added to retry/TTL timing so thousands of clients do not retry/expire simultaneously.
DLQ	After max retries, move poison jobs to a dead-letter queue for investigation rather than blocking the main queue.
Lease	Time-bounded ownership. If owner dies and does not renew, ownership automatically becomes available again.
Distributed lock	Coordinates mutually exclusive access across processes. Prefer leases/conditional writes where possible to avoid indefinite ownership.
Heartbeat	Periodic "I am alive" signal. Missing several intervals suggests a host/worker is unhealthy.
Canary deployment	Deploy to a tiny representative subset, evaluate health, then expand only if safe.
Rollback	Restore previous known-good version/state when health checks fail.
Cache TTL	Maximum time cached data remains valid before expiration. Shorter TTL gives fresher data but more source load.
Cache stampede	Many clients miss the same key simultaneously and overload the source. Use single-flight/request coalescing, locks, or jitter.
Strong consistency	Reads reflect the latest successful write, usually with more coordination/latency.
Eventual consistency	Replicas may temporarily disagree but converge; often improves availability/latency.

Python Interview Syntax - 2-Minute Revision

```
from collections import Counter, defaultdict, deque
import heapq

# Frequency
freq = Counter(nums)

# Queue / BFS
q = deque([start])
x = q.popleft()
q.append(next_node)

# Min heap
heapq.heappush(heap, value)
smallest = heapq.heappop(heap)

# Max heap using negatives
heapq.heappush(heap, -value)
largest = -heapq.heappop(heap)

# Build heap in O(N)
heapq.heapify(heap)

# K largest using a key
heapq.nlargest(k, items, key=lambda x: score[x])

# Sort intervals
intervals.sort(key=lambda x: x[0])

# Four directions
for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
    nr, nc = r + dr, c + dc

# Safe dictionary lookup
value = mapping.get(key)

# Default list per key
history = defaultdict(list)
history[key].append((timestamp, value))
```

Final 5-Minute Checklist Before the Interview

- Never start coding immediately. Restate + clarify first.
- Always say brute force before optimal, even if briefly.
- Name the pattern explicitly: "This is topological sort", "This is multi-source BFS", etc.
- Do not claim a complexity without explaining what N/V/E/U/K means.
- For graph problems, define the direction of every edge out loud.
- For BFS, mark visited when ENQUEUING, not when dequeuing, to avoid duplicates.
- For heaps, say why min heap of size K is enough for Top K.
- For LLD, keep entities small and responsibilities single-purpose.
- Do not force design patterns. Explain why the chosen pattern helps extension.
- For fleet questions, remember: retries -> idempotency; worker crash -> lease/visibility timeout; repeated failure -> DLQ; mass rollout -> canary + health gate + rollback; resource reservation -> atomic conditional write/lease; stale cache -> TTL/invalidation.
- When stuck, communicate. A partially coded correct direction with clear reasoning is better than silent guessing.
- Use your strongest language. Clean Python is fine unless the interviewer explicitly asks for Java.

Final reminder:

The goal is not to memorize 15 answers word-for-word. Memorize the recognition trigger, the data structure, the correctness invariant, and the complexity. If the interviewer changes the story from courses to deployments or oranges to failure propagation, the underlying pattern stays the same.